

CS 576 – Assignment 2 Spring 2026

Assigned Monday 02/23/2026,

Solutions due on Friday 03/13/2026 at Noon 12:00 pm

Late Policy: None.

Total Points: 200.

This programming assignment will help you to understand the working of DCT and how it is used by standard compression algorithms like JPEG and MPEG. You will also create a content-adaptive encoder decoder which needs to function better than the standard JPEG compressor.

Content-Adaptive NxN Block Image Encoder/Decoder

Design and implement a still-image compressor like JPEG using a fixed 8x8 block based DCT. As an extension, adapt your setup to not use fixed 8x8 blocks, but variable sized NxN blocks depending on the content (N can change by region). Your codec must choose block sizes based on the image content and compress each block with a transform + quantizer process. The goal is to surpass baseline JPEG (quality factor tuned to match bitrate).

Your program will take as input 4 parameters and be invoked as

myProgram InputImage M Q B

where the parameters are defined as:

- *InputImage* – is the image to input to your coder-decoder (you may assume a fixed size and format that will be described on the class website).
- *Mode M* – This will take on values of M=1 for standard 8x8 DCT or M=2 for optimized sized blocks of NxN. Details explained below
- *Quantizer Q* – an integer that can have a value either -1 or positive. If positive, it will be used to quantize all the DCT coefficients by 2^Q . For ease of implementation, you are uniformly quantizing all DCT coefficients of all channels. If $Q = -1$, then you will need algorithmically find a value for Q to satisfy the requirements imposed by the next parameter B .
- *BitsPerPixel B* – a floating-point number which may be -1.0 (so that the Q can control quality of output) or may be a positive value > 0.0 . If so, your algorithm needs to compute a Q value that will get the average bits per pixel to match (approximately) B .

NOTE – Either Q or B will need to be -1, both can't take on positive values

You will need to implement a hotkey – letter “B” or “b” as a key board press which will toggle on/off the display of boundaries of all the 8x8 or NxN blocks. You will be evaluated on the map of how algorithms have arrived at NxN.

*You have two output requirements – The image needs to be displayed on screen after encoding/decoding. Please save your encoded file of quantized DCT coefficients in all three channels in a file named **InputImage.DCT**. We will use zip metrics on this file to quantifiably evaluate your compression output. Please refer to evaluation metrics below for the .DCT file*

Details for M :

For $M = 1$, use the standard JPEG pipeline technique where you divide each image channel into 8×8 blocks and perform a DCT on the block to get 64 coefficients for each block. These 64 coefficients are then quantized into integers by dividing each value by 2^Q . The decoding proceeds by multiplying the quantized values by 2^Q and then computing the IDCT for each channel.

For $M = 2$, you will need to compute how to divide the image into varying sized $N \times N$ blocks for a better compression system based on the content. You will need to follow these guidelines:

- Block sizes must be square ($N \times N$) and N must be selected from the set $\{2, 4, 8, 16, 32\}$ (extendable by you) or you may choose a non-power of two as long as constraints outlined here are satisfied.
- All blocks must be a square $N \times N$
- Block boundaries must tile the image (no overlapping blocks). You may allow small leftover strips. Your algorithm needs to find a way to edges appropriately
- The block-size decision may be made per-image or per-block, but you must document and defend the algorithm.

Details for Q :

This parameter helps assess quantization to quantize the DCT coefficients. It is normally given as an input for the encoder/decoder processes. The quantization process is achieved by dividing all DCT coefficients uniformly by a value 2^Q . While the value of Q may be given as an input to your process, you may also need to automatically compute it to achieve compression for a certain bpp (bits per pixel) value.

Details for B :

B signifies the average bits per pixel. Its value can be -1.0 or a positive floating-point value > 0.0 . If -1.0, it signifies that the bits per pixel are controlled by the quantizer Q value above and there is no need to do anything further here. Otherwise, you will need to compute what the Q value should be to achieve the suggested average bits per pixel B value. Normally, in Transform Coding Standards, the lossy transform output of quantized coefficients is then further lossless compressed using lossless compression processes like Huffman or Arithmetic Coding. In this assignment you have not been asked to implement the latter process, but to get to a fair compressed bits per pixel calculation we will use the following metric using the standardly available zip/gzip process.

$$\text{Compressed bits/pixel} = \text{zip}(\text{basefile.DCT}) / (\text{width} \times \text{height})$$

In other words, compute a value for Q , which when applied to the quantization process will generate a file of quantized DCT coefficients, whose size after zipping and dividing by the number of pixels will yield a value that is equal to or less than B .

Required System Components

Block Decision Module:

- Algorithm that subdivides the image into a tiling of $N \times N$ blocks. For $M=1$, N is fixed to be 8, but for $M=2$, N is variable and can take on any number of values. The final map of $N \times N$ blocks could be greedy, quadtree, graph-cut, dynamic programming, or content-driven heuristic, but it must be
- We need to see the output map of $N \times N$ blocks with the required keyboard toggle “B” or “b” key press.

Transform & quantization per block:

- Your transform computation computes DCT for an $N \times N$ block.
- Post the DCT computation, each frequency coefficient is then quantized by 2^Q into appropriate data types.

Display and File storage:

Your process will need to generate two requirements depending on the input parameters

- An image displayed on screen after DCT conversion, quantization, inverse quantization and IDCT using all three channels.
- A file `basename.DCT` which is the direct save of your block by block quantized coefficients. You are not required to perform any entropy coding (you may if you want to).

Evaluation of Compression Metrics using your .DCT file:

Normally, in Transform Coding Standards, the lossy transform output of quantized coefficients is then further losslessly compressed using lossless compression processes like Huffman or Arithmetic Coding. In this assignment you have not been asked to implement the latter process, but to get to a fair compressed bits per pixel calculation we will use the following metric using the standardly available zip/gzip process.

$$\text{Compressed bits/pixel} = \text{zip}(\text{inputfile.DCT}) / (\text{width} \times \text{height})$$

1. *myProgram Example.rgb 1 0 -1*

Here you are encoding *Example.rgb* with $M=1$ (8x8 blocks), $Q = 0$ ($2^0=1$ which means no quantization), and no need for *B*. Your output will look like the input.

2. *myProgram Example.rgb 1 10 -1*

Here you are encoding *Example.rgb* with $M=1$ (8x8 blocks), $Q = 10$ ($2^{10}=1024$ which will be used to divide all the coefficients), and no need for *B*. Your output should look very lossy.

3. *myProgram Example.rgb 2 1 -1*

Here you are encoding *Example.rgb* with $M=2$ ($N \times N$ blocks), $Q = 0$ ($2^1=2$ which will be used to divide all the coefficients), and no need *B*.

4. *myProgram Example.rgb 1 -1 1.5*

Here you are encoding *Example.rgb* with $M=1$ (8x8 blocks), $Q = -1$ (Need to find a value for Q), and $B=1.5$ which means when you save out the quantized *Example.DCT* file and do a zip on it,

its file size/(number of pixels) which is the average bits/pixel should evaluate to 1.5 (approximately)

5. *myProgram Example.rgb 2 -1 1.5*

Here you are encoding *Example.rgb* with $M=2$ ($N \times N$ blocks), $Q = -1$ (Need to find a value for Q), and $B=1.5$ which means when you save out the quantized *Example.DCT* file and do a zip on it, its file size/(number of pixels) which is the average bits/pixel should evaluate to 1.5 (approximately). The expectation is that the quality of the output should be better than the above example 4. because of adaptive block sizes for the same bits per pixel.

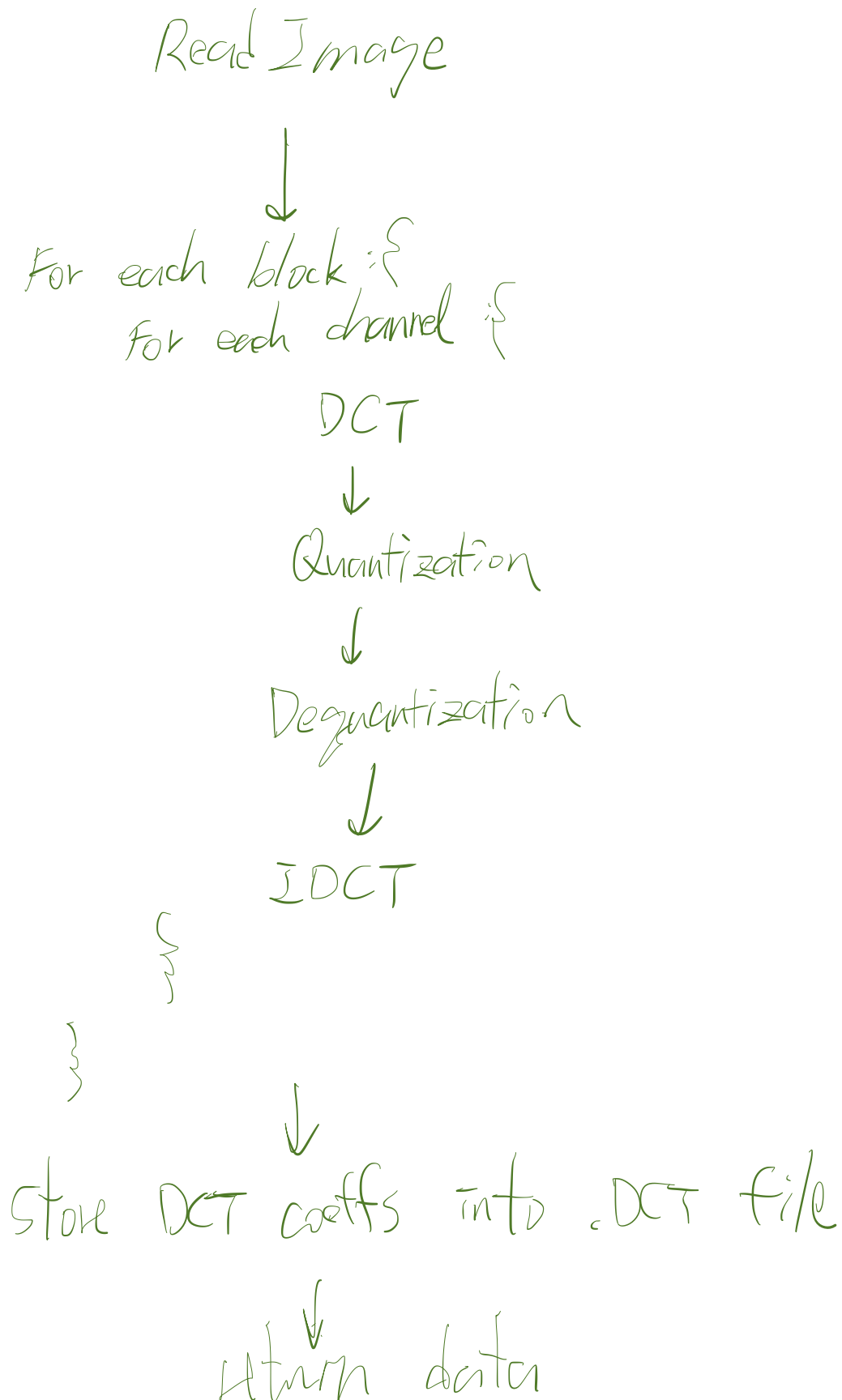
What should you submit?

- Your source code, and your project file needed to compile the code. Please do not submit any binaries or images. We will compile your program and run our cases accordingly. If you need to include a *readme.txt* file with any special compilation instructions, that is fine too.

Divide this assignment into 3 phases:

1. Fixed size pipeline ($M=1$)
2. Adaptive size pipeline ($M=2$)
3. Optimal quantization mode (when $B > 0$ & $Q = -1$)

1. Fixed Pipeline



2. Adaptive Pipeline

Read Image



Compute luminance for adaptive processing

Compute the block map according to luminance value



The rest is the same

Compute block Map: 拿到 luminance buffer

後可以把它分成不同 block size, 計算

DCT coeff., 用 $\frac{\text{dc coeff. value}}{\text{total coeff. value}}$ 判斷是否需要把 block size 變更小 via recursion.

return an array of blocks

3. Optimal Quantization Mode

Compute DCT coeff,



Quantization with Q



store as a .DCT file



Compress to calculate bpp rate



See if bpp rate meet target bpp

Yes

No

Go to fixed/adaptive pipeline
w/ optimal Q value

Continue on Binary Search
with higher Q

Codes for this is going to be inserted into each pipeline

往下走到 pipeline 時記得因為 DCT coeff 已算过一次可以不用重算